<Draw It or Lose It>

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
2.0	04/04/26	JT Doyle	<Evaluation table>

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

CTS is helping The Gaming Room in taking their Android application, Draw It or Lose It, to a web-based application so it can be accessed on multiple platforms. The team must design a system that maintains the applications' performance and enables web-based systems using strict object management to ensure one instance is available in memory. Unique IDs and names will help validate that game, team, and player names are individual.

Requirements

- Web-based application available on multiple platforms
- Singleton Pattern
- Unique games, teams, players
- Support multiple users
- Efficiency in data handling and validation
- Provide a smooth real time experience

Design Constraints

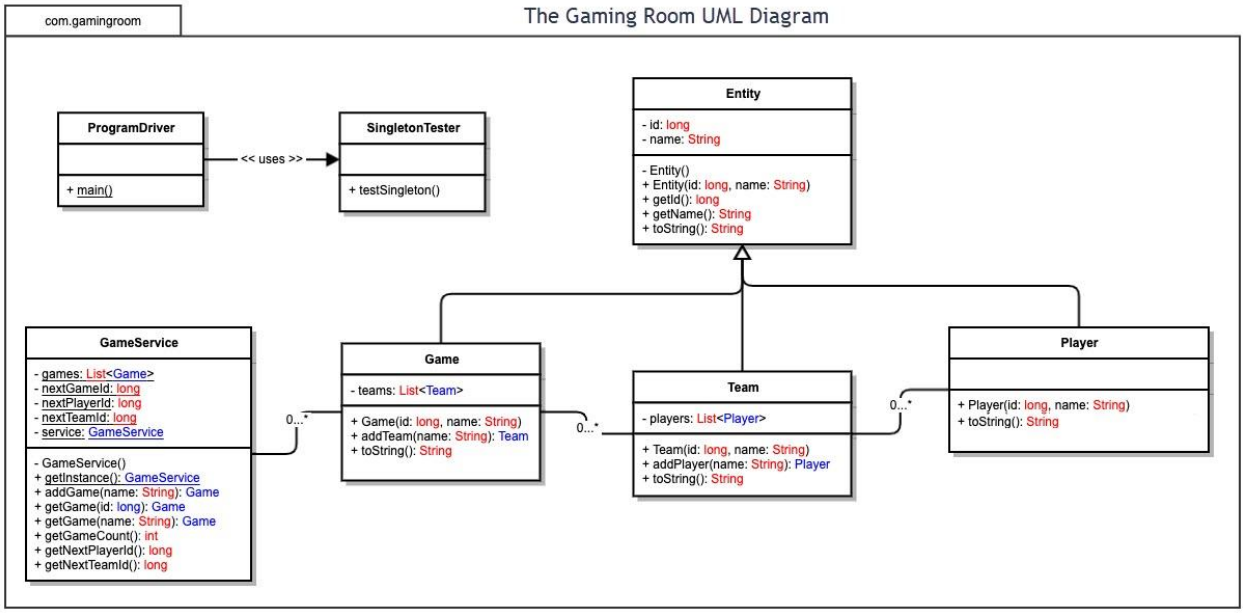
- Singleton System for single instances
- Unique names for validation
- Web-based application needs to support multiple users

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

Game, Team, and Player inherit from Entity allowing for shared properties like IDs to be defined once and accessed improving efficiency and consistency. Encapsulation of classes makes it easier to protect data integrity and system maintenance. Classes are modular with specific responsibilities to managing areas of the service. The Singleton pattern ensures that one instance is present in memory at a time.


com.gamingroom
The Gaming Room UML Diagram
ProgramDriver
+ main()
<< uses >>
SingletonTester
+ testSingleton()
Entity
- id: long
- name: String
- Entity()
+ Entity(id: long, name: String)
+ getId(): long
+ getName(): String
+ toString(): String
GameService
- games: List<Game>
- nextGameId: long
- nextPlayerId: long
- nextTeamId: long
- service: GameService
- GameService()
+ getInstance(): GameService
+ addGame(name: String): Game
+ getGame(id: long): Game
+ getGame(name: String): Game
+ getGameCount(): int
+ getNextPlayerId(): long
+ getNextTeamId(): long
0...*
Game
- teams: List<Team>
+ Game(id: long, name: String)
+ addTeam(name: String): Team
+ toString(): String
0...*
Team
- players: List<Player>
+ Team(id: long, name: String)
+ addPlayer(name: String): Player
+ toString(): String
0...*
Player
+ Player(id: long, name: String)
+ toString(): String

Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS is based on a UNIX system structure. It can use multithreading and multitasking. Handling multiple tasks is important for server environments. But systems like Linux and Windows are supposed to be better for scalability and thread management. macOS can support server functionality. Operating System Concepts, 8th Edition says flexibility and scalability are highly important for server systems, which may be limited depending on system design and resource management.	The textbook says Linux is designed for scalability and flexibility. It uses tasks instead of separating processes and threads, and its efficient resource sharing through things like the clone() system call. The inclusion of the Native POSIX Thread Library (NPTL) allows Linux to maintain compatibility very well across operating systems. Its scheduling system is priority-based and designed to scale efficiently, even with many tasks, which is essential for the web based applications.	Windows systems (such as Windows XP used in the text) supports server side operations through a structured thread and priority system of scheduling. Each thread is mapped to a kernel thread for efficient CPU use and response. The one-to-one thread model is easily managed but as the number of threads increases it becomes less efficient than many thread systems. The structured design is easily observed, but can be less flexible than Linux's task system.	Mobile operating systems support multithreading and multitasking, but the textbook says operating systems need to balance being responsive and effienctly use resources. Mobile systems are designed for interactive tasks not large scale server management. They also don't have the processing power to manage all the concurrent processing for server side applications. They are just too limited in scalability and resource management. Making them the worst of the options.
Client Side	Modern operating systems support user level and kernel level threads, allowing applications to be responsive. macOS is a UNIX inspired	Linux supports efficient multitasking and resource sharing. The ability to share resources between tasks using the clone()	Windows thread model and scheduling system is good for client side systems. The priority classes and dynamic priority	Mobile operating systems prioritize. The textbook says scheduling systems favor interactive processes, which mobile applications are pretty much

	system and supports the needed threading models. This would helps applications handle the user interactions efficiently. For client side use this means applications can remain responsive while performing background tasks. Since the application runs in a browser most client side operations is handled by the operating system.	system allows efficient simultaneous processes. The scheduler favors interactive tasks, which improve responsiveness for the users. Development would need to account for variations in how tasks and resources are managed across systems since they are handled differently.	adjustments ensures that interactive applications receive more CPU time, making applications more responsive. Considerations for development would be understanding how thread priorities and scheduling affect could effect application performance.	required to be. Mobile systems need to carefully manage CPU time, memory, and battery usage. Mobile systems just make these things more complicated. Applications need to be optimized for performance and efficiency. Developing for multiple device types increases development time due to differences.
Development Tools	Mac OS uses higher level programming languages so it is easy to build an application. Since it has a layered design and is modular, applications don't have to focus too much on low level processes of the system but the developer needs to understand how the OS operates differently.	Linux is very flexible and has standardized POSIX with NPTL. So it is simpler developing programs to handle many simultaneous tasks.	Windows development uses the Win32 API and thread libraries. These tools allow developers to create manage threads and scheduling more easily. But without understanding of the systems this can make it more complex to develop.	Mobile development has limited CPU and memory so thread use and scheduling needs to be much more deliberate and efficient compared to the rest.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2008). *Operating system concepts* (8th ed.). Wiley.

<u>Recommendations</u>

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: The recommended operating platform for Draw It or Lose it is a Linux server environment. Linux is the recommended choice because it gives the best options to be scalable, changed drastically when needed, and can handle a large user base concurrently. These features will be essential for a live multiplayer web service. Linux also provides options for cloud environments that will make it easier to operate across multiple system types and improve scalability on an as needed basis; or Linux's Native POSIX Thread Library (NPTL) can be used to maintain compatibility very well across operating systems. Linux provides better server side operation compared to the next two best options in Windows and MacOS.

2. **Operating Systems Architectures**: Linux uses modular monolithic kernel architecture, where operating system design core services like memory management, drivers, and storage management all run within one large, privileged kernel address space, resulting in higher performance due to low overhead for communication between components. This makes it both efficient and flexible. It supports both processes and threads using a clone() system call that allows for sharing resources between tasks when needed for even more efficient processes, especially when handling multiple users at once. It has a priority based scheduling system that can ensure the critical processes get the proper CPU usage needed. This combined with preemptive multitasking Linux offers ensures CPU usage isn't over invested into individual operations. Linux will be great for managing real time processes with many users while maintaining performance and stability.

3. **Storage Management**: A Linux file system can be used in combination a relational database system like MySQL. The database would hold structured data for things like individual accounts, players, teams, and games. While the linux file system will manage the application files and game history.

4. **Memory Management**: Linux has options for memory management like Virtual Memory, giving the illusion of segmented memory space and allowing more processes to run concurrently by swapping inactive memory portions to disk. It also uses Paging where physical and virtual memory are divided into "pages", small sets of fixed memory space, that then can be placed into "page tables" to map the physical and virtual memory for optimized resource usage. Like most popular OS there is Kernel space and User space to separate core OS functions for secure and stable memory. There is also memory caching, typically through using the page system so frequently used pages are readily accessed for efficiency. This is important for Draw It or Lose It because it helps the system run multiple game sessions at once while keeping gameplay smooth and responsive for users.

5. **Distributed Systems and Networks**: Since Draw it or Lose It is needing to communicate across platforms a client server model and cloud based architecture should be used. The clients would connect to a centralized set of servers, the number of these servers can be increased or decreased to meet the demand of user volume. These servers would process the games and manage data, while communications happen via HTTP or HTTPS protocols. Servers can have issues though so there needs to be back up systems or use cloud infrastructure or both.

6. **Security**: Security needs to be very high priority because there will be a large user base on many different platforms making more vulnerabilities than if it was just operation on one platform.

Additionally the trust of the users is important for public perception. The linux based system would allow for a permissions system and tagging to ensure data is only accessible when requested by the proper source. File access control ensures critical files are harder to access and manipulate maliciously. And an OS firewall provides additional protection. To protect the communication from client to server encryption through HTTPS, not HTTP, should be used to transmit data over networks. Security authorization with users, passwords, and multifactor authentication should be incorporated. Pass words should be hashed with a private hash operation and stored in a secure database. Validating server side operations should also be used not just from the client side to prevent malicious activity. All of this along with regular updates to be proactive on fixes to be ahead of security vulnerabilities.

Resources

Silberschatz, A., Galvin, P. B., & Gagne, G. (2008). *Operating system concepts* (8th ed.). Wiley.

Molochko, A. (2024, March 18). *Why is Linux a monolithic kernel?* Baeldung.

Sevak, M. (2021, February 6). *Linux: A fully preemptive operating system*. LinkedIn.

LambdaFunc. (2024, November 15). *Linux memory management*. Medium.